ParkShare

Park smarter. Share more.

Payments Architecture Plan

Stripe Connect Custom + separate charges and transfers

2026-10-10

www.park-share.ro

Table of Contents

1 Purpose and Summary 3
2 Locked Decisions 3
2.1 Commission model 3
3 Why Stripe Connect Custom 3
4 The One Big Custom Trade-off 4
5 Architecture Overview 5
5.1 Money flow (Model A: separate charges and transfers) 5
5.2 Charge type: separate charges and transfers 5
6 Host Connected-Account Model (Accounts v2) 6
7 Who needs what 6
8 Identity Verification (KYC) Requirements 7
9 Legal Prerequisites (Custom-specific, non-negotiable) 7
10 Data Model (Prisma) 7
11 Backend (Express) API 8
11.1 Host onboarding 8
11.2 Driver 8
11.3 Host wallet 8
11.4 Webhooks 8
11.5 Background jobs 8
12 Frontend Flows 8
12.1 Host onboarding wizard (inside ParkShare) 8
12.2 Host earnings and wallet 9
12.3 Driver 9
13 Fees 9
14 Compliance Checklist 9
15 Milestones 9
16 Testing 10
17 Decisions to Confirm 10
18 References 10

1 Purpose and Summary

This document specifies how ParkShare will process **real money** end to end: drivers pay by card, funds settle to the ParkShare company account, hosts receive their share in an in-app wallet, and hosts withdraw to their bank account by IBAN. It is a review plan, not final code, and it ends with the decisions that must be confirmed before implementation begins.

The design uses **Stripe Connect** with **Custom** connected accounts and the **separate charges and transfers** charge type. Drivers never touch Stripe, and hosts never see a Stripe account or dashboard.

> **Two sentences to remember:** Stripe is the card processor and the money-movement engine. ParkShare is the platform (merchant of record); the webhook from Stripe — never the app — is the source of truth for whether money actually moved.

2 Locked Decisions

- **Charge type:** separate charges and transfers — the charge lands on the ParkShare account first; the host's share is transferred to the host's connected account afterwards.
- **Merchant of record:** ParkShare. We are responsible for Stripe fees, refunds and chargebacks.
- **Commission:** 20%. By default the **host nets 20% less** (the driver pays the host's listed price; the host receives 80% of it). Configurable via `COMMISSION_RATE` and `PRICING_MODE` (`HOST_NET` – default – or `DRIVER_GROSS`).
- **Drivers:** native Stripe PaymentSheet. No account. Stripe is essentially invisible.
- **Hosts:** Custom connected accounts — no Stripe account, no Stripe login, no Stripe dashboard, ever. All onboarding, earnings and support live inside ParkShare.

2.1 Commission model

Item	Driver pays	ParkShare keeps	Host receives
Listed price: 10.00 RON	10.00 RON	2.00 RON (20%)	8.00 RON (80%)
Listed price: 25.00 RON	25.00 RON	5.00 RON (20%)	20.00 RON (80%)

Why "host nets 20% less" (and not "driver pays 20% more"):

- It is an unambiguous **20% of every transaction** (the "driver pays more" framing is only 16.7% of the gross).
- The driver pays exactly the price they see — no checkout surprise, which is the number-one trust problem in a marketplace.
- It matches the logic already in the codebase (`providerCut = finalPrice * 0.9` becomes `* 0.8`).

> **Disclosure is mandatory on both sides.** Drivers see the total they pay. Hosts see, at the moment they set a price and on every booking, `Gross 10.00 RON -> You earn 8.00 RON (after 20% ParkShare fee)`. If a later review prefers the "driver pays more" model, it is a single configuration flip — the rest of this plan is unchanged.

3 Why Stripe Connect Custom

Regular payouts to a third party's bank account are legally impossible without Know Your Customer (KYC) verification. The question is not **whether** verification happens, but **who owns the experience**. Three options exist:

Type	Host experience	Sees Stripe?	Build effort
Express	Stripe-hosted onboarding; optional Stripe Express dashboard	Yes	Low
Custom	**Fully white-labeled inside ParkShare; no dashboard**	**No**	**Medium**
Standard	Host signs up for a real Stripe account	Yes	Low

Custom is chosen because the host never sees Stripe branding or a Stripe login. The cost is that Custom means **the platform owns everything**: onboarding, the earnings/withdrawal UI, reporting, communications, and support.

4 The One Big Custom Trade-off

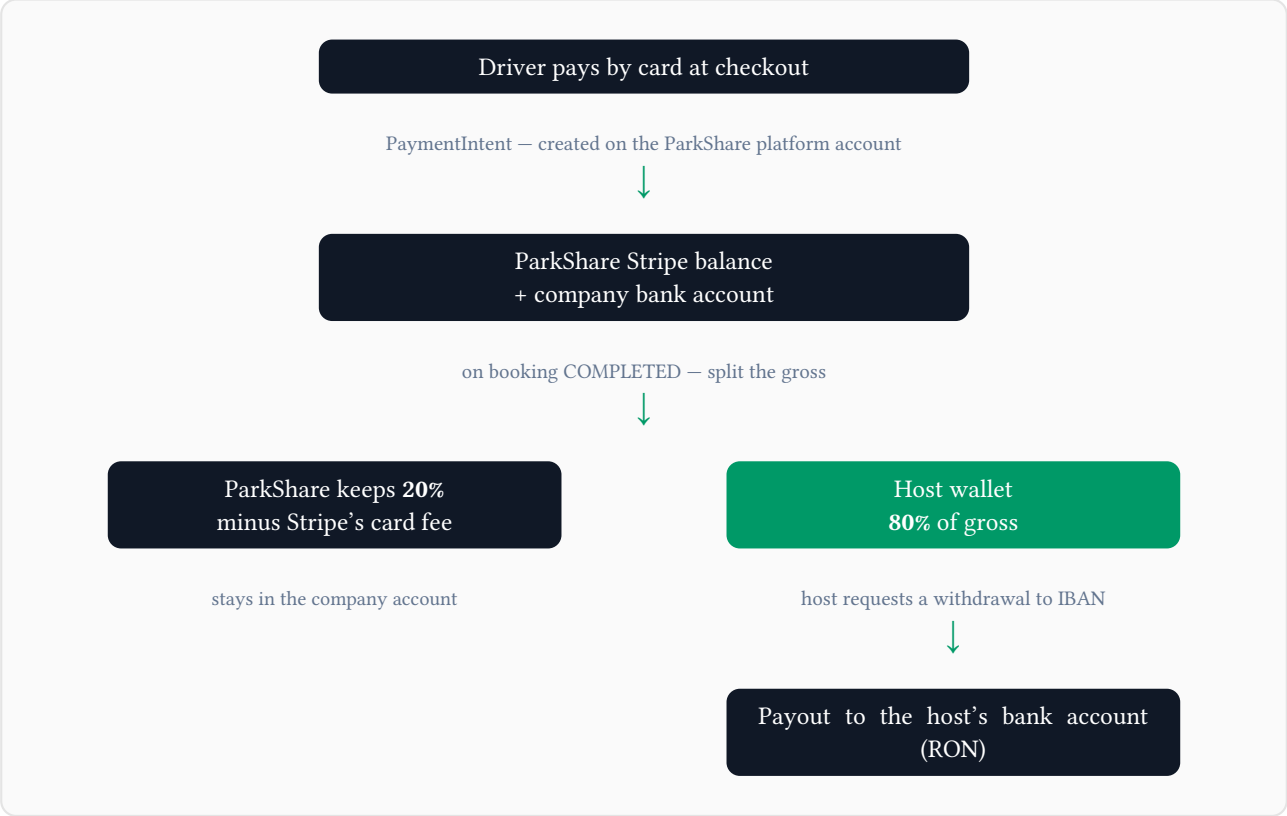
Because Custom accounts have no Stripe dashboard, Stripe does not collect KYC for the host by default. There are two sub-modes; this is the key decision to make.

Aspect	2A. Custom + Stripe-hosted (co-branded) onboarding	2B. Custom + fully in-app (API) onboarding
KYC form	Stripe-hosted page styled with ParkShare logo/color	Built field-by-field inside ParkShare
Host sees Stripe?	A co-branded form; never a dashboard	Nothing at all
Who collects requirements	Stripe (`dashboard: "none"`)	ParkShare (`losses_collector: "application"`)
Negative-balance liability	Stripe	ParkShare
Build effort	Low	High
Ongoing compliance upkeep	Stripe keeps requirements current	ParkShare tracks `requirements` forever

> **Recommendation: start with 2A.** It delivers "no Stripe account / no dashboard for the host" immediately, with a fraction of the work. Move to 2B only if the co-branded form is not neutral enough. The backend and frontend barely differ between the two.

5 Architecture Overview

5.1 Money flow (Model A: separate charges and transfers)



- Stripe's processing fee (roughly 1.5% + fixed for EEA cards) is debited from ParkShare's account, so ParkShare absorbs it out of its 20%.
- The security deposit is a separate **manual-capture hold** and is not part of the 80/20 split.

> **Reality check.** The host's 80% does not literally sit in the company bank account until withdrawal. It moves to the host's Stripe connected account, and Stripe pays it to the IBAN on request. This is what keeps ParkShare compliant while preserving the "wallet + IBAN" experience. Parking third-party funds in the company account until withdrawal would make ParkShare a money-holder subject to licensing — avoid it.

5.2 Charge type: separate charges and transfers

Charge type	How funds move	Why / why not
Destination charges	Charge on platform, funds immediately routed to the host, platform keeps an application fee	Simplest split, but money does not "land in ParkShare first"
Separate charges and transfers	**Charge on platform first; a separate transfer moves the host's share; platform keeps the difference**	**Matches the required model; ParkShare controls timing and can hold funds**

We use **separate charges and transfers** because it matches the requirement that the money lands in the ParkShare account first, and because it lets us hold the host's share until a booking is completed (and until the host is verified).

6 Host Connected-Account Model (Accounts v2)

Each host is created as a **v2 Account** with the following configuration:

- `identity: country: "ro", entity_type: "individual"` (or `company` for business hosts), plus `business_details`.
- `dashboard: "none"` — the host never sees a Stripe dashboard.
- `defaults.currency: "ron"`.
- The `recipient` configuration with the `stripe_balance.stripe_transfers` capability, so the account can receive transfers; plus a payout method (external IBAN) so funds can leave to the bank.
- Responsibility mode per the sub-mode choice:
  - **2A:** `defaults.responsibilities.losses_collector: "stripe"` -> Stripe collects KYC (`requirements_collector = stripe`).
  - **2B:** `defaults.responsibilities.losses_collector: "application"` (and `fees_collector: "application"`) -> ParkShare collects KYC.

> **Validate before committing.** Confirm with Stripe that a **Romania-based platform** may create **Custom / `dashboard: "none"`** accounts — Custom availability varies by platform country, and Stripe may steer you toward Express. Verify this in a sandbox and with Stripe support first; it is the only true potential blocker in this plan. If Custom is unavailable for RO, fall back to Express or to v2 accounts with `dashboard: "none"`.

7 Who needs what

- Drivers (payers) — no ID. Only contact + card details (via native PaymentSheet). No passport/ID upload, ever.
- Hosts (payees) — only when they want to receive money. When a host first tries to list a spot and earn (or to enable withdrawals), we must collect the minimum information to legally pay them out. Under EU/Romanian AML rules, the paying party must identify the payee. This is standard (e.g. Airbnb host, Uber/Bolt driver, eMAG Marketplace seller).

Just-in-time, not at signup:

- Don't ask for payouts info during registration. Allow browsing, booking, and creating a draft spot without it.
- The first time they go to **Add/Edit Spot** → try to earn, or to **Wallet/Withdrawals** → "Set up payouts", trigger the onboarding.
- Only ask for what's `currently_due` at that moment (often name + DOB + address + IBAN; ID/selfie only if explicitly requested). Don't pre-collect every possible field.

Where it lives in the app:

- **Settings → Personal Information** — prefill basic legal name/address/phone (mostly already present). Can extend slightly if needed.
- **Account → Wallet / "Payouts and Wallet"** — becomes the entry point to "Set up payouts" / see status (Pending/Action needed/Active).
- Existing screens: `frontend/src/screens/settings/PersonalInformationScreen.tsx` (name/photo/email), `frontend/src/screens/settings/PaymentMethodsScreen.tsx` (cards, to be reworked to "Wallet" + payout setup).

8 Identity Verification (KYC) Requirements

For a Romanian **individual** host receiving payouts:

- **Basic info:** legal name, date of birth, full address, email, phone (prefill from Personal Information where available).
- **Identity document:** only if the provider marks it as required (e.g. front/back of a government-issued ID, and possibly a liveness/selfie). Do **not** require it by default.
- **Tax identifier:** `id_number` / CNP if requested.
- **Bank account:** IBAN (RON domestic) — required to withdraw.
- **ToS acceptance:** record IP and timestamp. In 2A (Stripe-hosted co-branded onboarding), Stripe handles this; in 2B (fully in-app), ParkShare must.

For a `company` host: business name, `CUI`, registered address, representative, and UBOs/directors holding 25% or more (only if payouts will go to a company).

> **Collect minimums. Defer until needed.** Start by creating the Stripe Connect account (or returning its `requirements.currently_due`) only when the host actually needs payouts. If the provider returns an empty `currently_due`, show "Payouts ready" and don't ask for anything. Always surface `requirements.past_due` / `future_requirements` and block withdrawals until `payoutsEnabled` is true.

9 Legal Prerequisites (Custom-specific, non-negotiable)

Stripe's own requirements for Custom accounts:

1. **API version** at least `2014-12-17` (we will pin a current version).
2. **Updated Terms of Service** that reference Stripe's services agreement. This needs legal review — a lawyer should update the ParkShare ToS.
3. **Handling information requests:** when Stripe needs data it asks the platform, and the platform must collect it from the host, or Stripe may disable payouts.
4. Because hosts cannot log in, **ParkShare builds** onboarding, the earnings/withdrawal UI, reporting and support communications.

10 Data Model (Prisma)

- **User (host):** `+stripeAccountId`, `+connectStatus` (NONE | PENDING | ACTIVE | RESTRICTED | REJECTED), `+payoutsEnabled`, `+requirementsDue` (Json), `+defaultCurrency`.
- **Spot:** `price` = the listed driver price per hour.
- **Reservation:** `+baseAmountMinor`, `+commissionMinor`, `+hostNetMinor`, `+driverTotalMinor`, `+currency` ("RON"), `+paymentIntentId`, `+transferId`, `+chargeStatus`.
- **LedgerEntry** (source of truth for the wallet): `hostId`, `reservationId?`, `type` (EARNING | PAYOUT | REFUND_CLAWBACK | CHARGEBACK_CLAWBACK | ADJUSTMENT), `amountMinor` (signed), `currency`, `status`, `providerRef`, `createdAt`.
- **Payment** (driver charges): `userId`, `reservationId?`, `amountMinor`, `currency`, `paymentIntentId`, `status`.
- **Withdrawal:** `hostId`, `amountMinor`, `currency`, `status` (REQUESTED | PROCESSING | PAID | FAILED), `stripePayoutId?`, `ibanLast4`, `bankName`, `createdAt`.
- **WebhookEvent:** `stripeEventId` (primary key), `type`, `processedAt` — for idempotency.
- **All money stored as integer minor units (bani).** No floats.

11 Backend (Express) API

11.1 Host onboarding

- `POST /connect/accounts` — create or refresh the host's v2 account; return current `requirements`.
- `POST /connect/account-link` — **2A only:** return the Stripe-hosted co-branded onboarding URL.
- `POST /connect/account` — **2B:** accept collected KYC fields, IBAN and ToS acceptance and push them to Stripe.
- `GET /connect/status` — `payoutsEnabled`, `requirementsDue`, `connectStatus`.

11.2 Driver

- `POST /payments/booking-intent` — the server computes `{base, commission, total}` and creates a PaymentIntent; returns `clientSecret` plus the itemised breakdown.

11.3 Host wallet

- `GET /host/earnings` — ledger entries.
- `GET /host/wallet` — available and pending balances.
- `POST /host/withdrawals {amountMinor}` — validate ledger balance and `payoutsEnabled`, then create a Stripe payout to the IBAN.
- `GET /host/withdrawals` — history.

11.4 Webhooks

Endpoint `POST /webhooks/stripe`, whose raw body must be mounted **before** `express.json()` or signature verification will fail.

- `payment_intent.succeeded` — mark the booking paid.
- `account.updated` / `capability.updated` — sync `connectStatus`, `requirementsDue`, `payoutsEnabled`; notify the host.
- `transfer.created`, `payout.paid` / `payout.failed`.
- `charge.refunded`, `charge.dispute.created` — claw back the host's share.
- All handled idempotently via the `WebhookEvent` table.

11.5 Background jobs

- On booking `COMPLETED`: create a `Transfer` of `hostNetMinor` to the host account, and write a `LedgerEntry` of type `EARNING`. If the host is not yet verified, hold the earning as pending until active.
- Auto-complete stale reservations (the codebase currently has no scheduler).
- Do not transfer before the underlying charge funds are available in the platform balance.

12 Frontend Flows

12.1 Host onboarding wizard (inside ParkShare)

1. "Set up payouts" screen: explains payouts are to IBAN, ParkShare keeps 20% fee. Shown **only when the host needs to earn/withdraw** (just-in-time). Don't prompt at signup.
2. Prefill legal name/address/phone from **Settings → Personal Information** (`PersonalInformationScreen.tsx`) where possible.
3. Collect only what's currently required (**2B**) or open the Stripe-hosted co-branded form (**2A**). If nothing is required, skip the form.
4. Add IBAN (RON) and accept terms (as required). Record IP/timestamp if in-app.
5. Status screen: Pending / "Action needed" (show `requirements.past_due`) / Active. Driven by webhooks. Withdrawals disabled until `payoutsEnabled` is true.

12.2 Host earnings and wallet

- Add/Edit Spot: a live label showing the host's net, for example "You earn 8.00 RON/hr (after 20% fee)".
- Booking and earnings list: `Gross 10.00 -> Net 8.00`.
- **Wallet:** Available, Pending, and a **Withdraw** action (to the IBAN), plus history. The Withdraw button stays hidden until `payoutsEnabled` is true.
- Account tab: a **Payouts and Wallet** entry.

12.3 Driver

- A native PaymentSheet at checkout. The booking modal and receipt show the total. Remove the fake 1.5-second delay and the hardcoded "Visa 4242" as part of this work.

13 Fees

- Card processing fee (EEA card, roughly 1.5% + fixed) — absorbed by ParkShare out of its 20%.
- Connect, when the platform handles pricing: roughly **10 RON per active host per month** plus **0.25% + 1.50 RON per payout** (indicative; confirm current RO pricing).
- Instant payout to a card: roughly 1% (optional feature).
- ParkShare is debited for refunds and chargebacks (merchant of record).

14 Compliance Checklist

- [ ] Update the ParkShare Terms of Service to reference Stripe's Services Agreement (lawyer).
- [ ] Record Stripe ToS acceptance (IP and timestamp) if using 2B.
- [ ] Confirm Custom availability for a Romania-based platform with Stripe.
- [ ] Fiscal receipts / e-Factura and VAT on the 20% commission for ANAF (accountant).
- [ ] GDPR: store only IBAN last-4 and Stripe references; never full bank data.
- [ ] Gracefully handle `restricted` accounts (payouts disabled) in the UI and by email.
- [ ] Define the negative-balance policy: allow a host to go negative on refund/chargeback, or block their next earnings.

15 Milestones

#	Deliverable	Real money?
0	Integer bani + ledger + pricing engine + host net-earnings UI (simulated)	No
1	Driver charges via PaymentSheet, into the ParkShare account	Yes
2	Custom accounts + host onboarding (2A hosted) + IBAN	Yes
3	Transfer on completion + wallet/earnings UI	Yes
4	Withdrawals / payouts + history	Yes
5	Refunds, chargebacks, clawbacks, reconciliation, stale-booking job	Yes
6	Passes as subscriptions; deposits as manual-capture holds	Yes
7	Fiscal / VAT integration and reporting	Yes

The minimum for "real money in, host paid to IBAN" is milestones **0 through 4**.

16 Testing

Use the Stripe **sandbox** and forward events locally with `stripe listen --forward-to localhost:8745/webhooks/stripe`. Use Stripe's test identifiers for identity and bank verification, and simulate `requirements.past_due`, restricted accounts, failed payouts, refunds and disputes. Secrets: `STRIPE_SECRET_KEY` and `STRIPE_WEBHOOK_SECRET` stay server-side; only `EXPO_PUBLIC_STRIPE_PUBLISHABLE_KEY` reaches the client.

17 Decisions to Confirm

1. **2A (co-branded hosted onboarding, fast) or 2B (fully in-app, maximum white-label)?** Recommendation: 2A first.
2. Confirm **Custom is allowed for a Romania-based platform** (verify in sandbox).
3. **Transfer timing:** on COMPLETED (recommended) or immediately at booking?
4. **Minimum withdrawal** and payout schedule (on-request only, or automatic daily)?
5. **Negative balance:** allow a host to go negative on refund/chargeback, or block their next earnings?
6. Keep the commission default "hosts get 20% less", or switch to "drivers pay 20% more"?

18 References

- Stripe Connect, Custom accounts: docs.stripe.com/connect/custom-accounts
- Stripe Connect, Accounts v2: docs.stripe.com/connect/accounts-v2
- Separate charges and transfers: docs.stripe.com/connect/separate-charges-and-transfers
- Place a hold on a payment method: docs.stripe.com/payments/place-a-hold-on-a-payment-method
- Expo, Stripe React Native SDK: docs.expo.dev/versions/latest/sdk/stripe